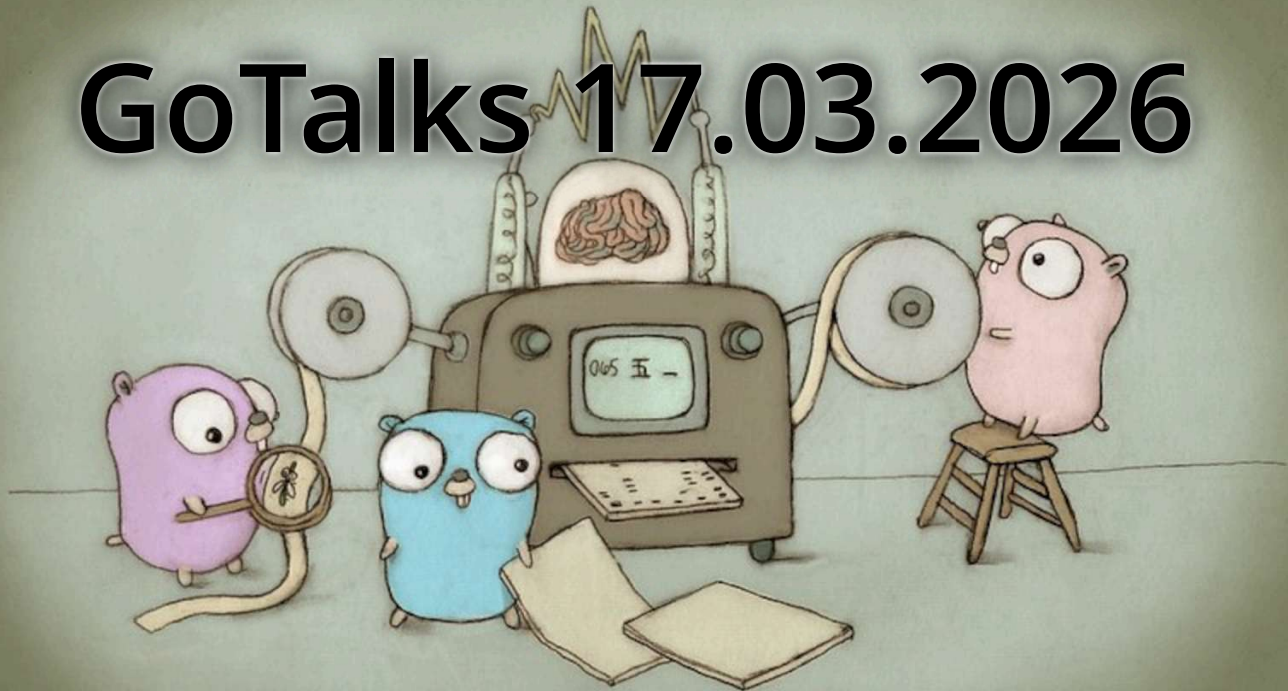


# GoTalks 17.03.2026





# How to reach us

---



[meetup.com/Golang-ZG](https://meetup.com/Golang-ZG)



[@golangzg](https://www.youtube.com/@golangzg)



[github.com/golang-zg/meetups](https://github.com/golang-zg/meetups)



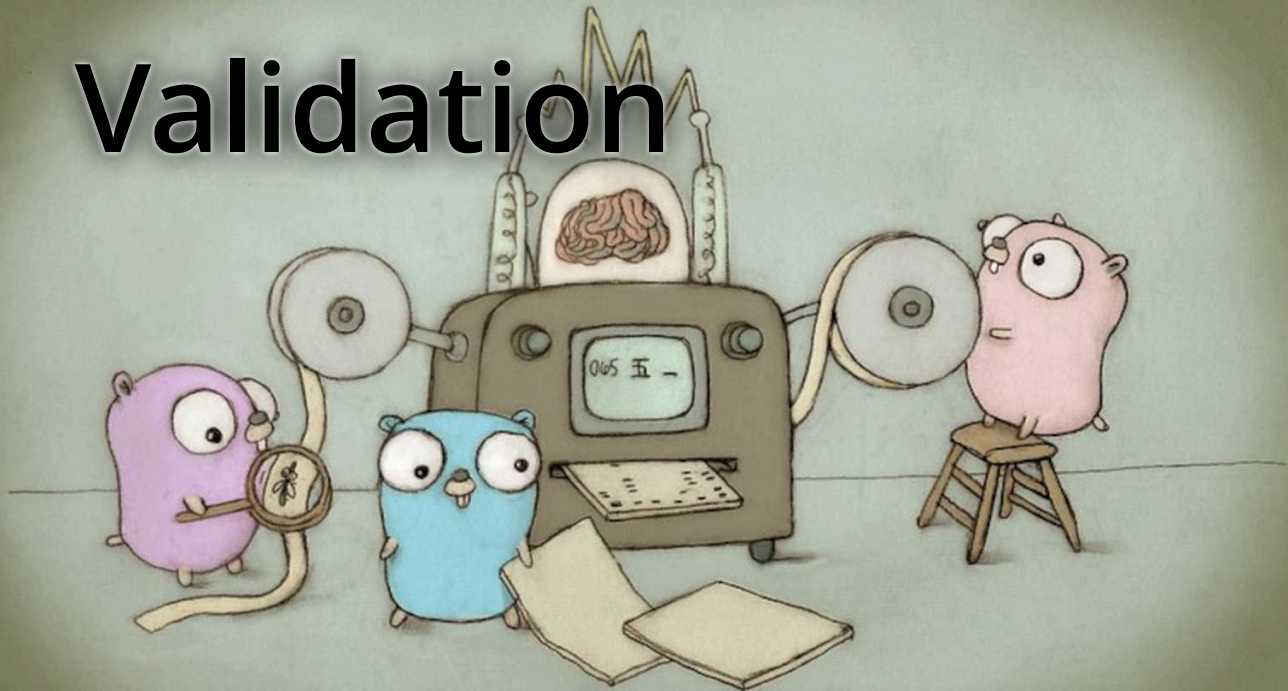
[@golangzg.bsky.social](https://bsky.app/@golangzg.bsky.social)



[invite.slack.golangbridge.org](https://invite.slack.golangbridge.org)



# Validation



What “validation” means in Go?

Validation usually refers to checking that:

- a struct's fields meet constraints (length, range, format)
- user input is safe and well-formed
- data is consistent before saving or processing

Official Support for Validation in Go:

- No official validation package in the standard library

# Validation types

---

- Manual validation
  - This is the idiomatic, built-in way. Write your own checks.



# Validation types - struct-tag

- Struct-tag validation
  - Use struct tags to specify rules, and a library to enforce them
  - [go-playground/validator](https://go-playground.com/validator) is a popular choice.

```
type User struct {  
    Email string `validate:"required,email"`  
    Age   int    `validate:"gte=18,lte=99"`  
}
```

- Custom validation functions

```
validate.RegisterValidation("is-something", func(fl validate.FieldLevel) bool {  
    return fl.Field().String() == "something"  
})
```



# Validation types - OpenAPI

---

OpenAPI (v2 or v3) describes your API using Schema based rules. These rules define what is valid input/output for each endpoint.

- language agnostic standard for API contracts.
- code generators (e.g., go-swagger, [oapi-codegen](#))
- schema validators (e.g., go-openapi/validate)
- runtime Schema validation

These tools read your OpenAPI spec and automatically enforce constraints.





# Validation types - OpenAPI

---

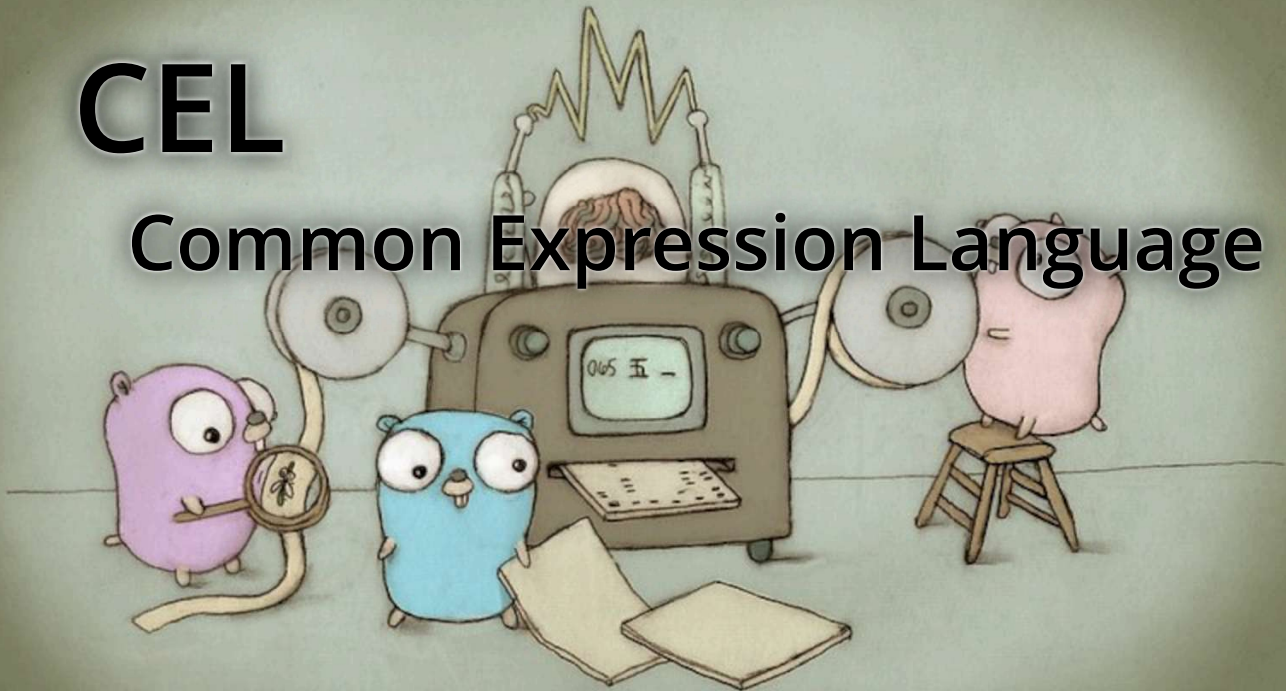
How this differs from manual validation?

- declarative and standardized across languages, while manual validation is imperative and specific to your codebase.
  - multiple libraries and tools available for many languages (both client and server-side).
- often used for API input/output, while manual validation can be used anywhere in your code.
- can be more comprehensive for API contracts, while manual validation is more flexible for complex logic.



# CEL

## Common Expression Language



# Validation types - CEL

---

- [cel.dev](https://cel.dev)
- ideal for performance-critical applications because it was designed to evaluate safely and quickly (nanoseconds to microseconds) with predictable costs
- CEL is designed for expressive, logic-based constraints that go beyond what OpenAPI keywords like `minimum`, `maximum`, `enum` or `pattern` can express.
- CEL is a small, embeddable expression language used to write boolean validation rules.
  - Kubernetes uses OpenAPI v3 structural schemas for CRDs, but adds CEL as an additional validation mechanism for complex rules.



# Validation types - CEL - example

```
applicant:
  celValidations:
    - name: "email_basic"
      expression: >
        email.contains("@")
      message: "Email must contain @"

    - name: "percentage_passed_check"
      expression: >
        percentagePassed > 50 && percentagePassed <= 100
      message: "Percentage passed must be 50 < x <= 100"
```



# Validation types - CEL - example

```
env, err := cel.NewEnv(  
    ext.NativeTypes(reflect.TypeOf(time.Time{})),  
    ext.Strings(),  
    ext.Math(),  
    cel.VariableDecls(  
        decls.NewVariable("email", cel.StringType),  
        decls.NewVariable("percentagePassed", cel.IntType  
    ),  
)
```

# Validation types - CEL - example

```
for _, rule := range schema.CELValidations {
    ast, issues := env.Compile(rule.Expression)
    if issues != nil && issues.Err() != nil {
        return issues.Err()
    }

    prg, err := env.Program(ast)
    if err != nil {
        return err
    }

    out, _, err := prg.Eval(map[string]any{
        "email":          a.Email,
        "percentagePassed": a.PercentagePassed,
    })
}
```



# Validation types - CEL - example

```
applicant := Applicant{  
  Email:      "test@example.com",  
  PercentagePassed: 50,  
}
```

output:

```
Validation failed:  
- Percentage passed must be  $50 < x \leq 100$ 
```



# Validation types - CEL - advanced example

- OpenAPI v3 + CEL
  - Kubernetes CRD validation

```
// +kubebuilder:validation:XValidation:rule= \
//     "!has(self.spec.config.default_path)", \
//     message="spec.config.default_path is set by ..."
```

```
type TCPService struct {
    Name string `json:"name"`
    // +kubebuilder:validation:Maximum=65535
    // +kubebuilder:validation:Minimum=1
    Port int `json:"port"`
}
```





# Validation types - CEL - advanced example

- OpenAPI v3 + CEL
  - Kubernetes CRD validation

```
// +kubebuilder:validation:Required  
Name      string          `json:"name"`
```

```
// +kubebuilder:validation:Enum=RSA;ECDSA;  
Keytype   string `json:"keytype,omitempty"`
```

```
// +kubebuilder:validation:Pattern=`^[^\s]+$`  
ID *string `json:"id,omitempty"`
```



# Validation types - CEL - advanced example

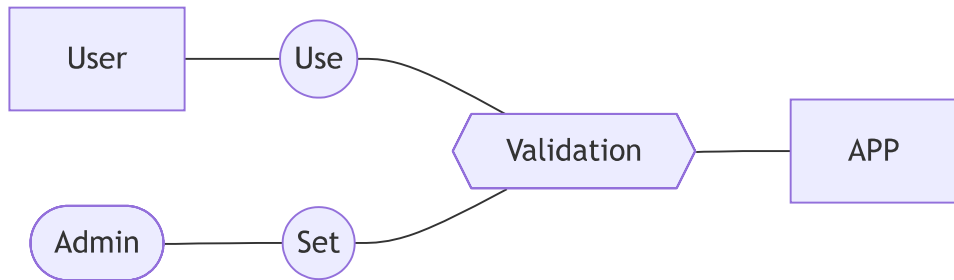
---

- normal way:



# Validation types - CEL - advanced example

- *No two companies are alike*
- Administrator sets rules, and users use the system according to those rules.
- Validation logic is decoupled from the application code and can be updated.



# Validation types - CEL - real world

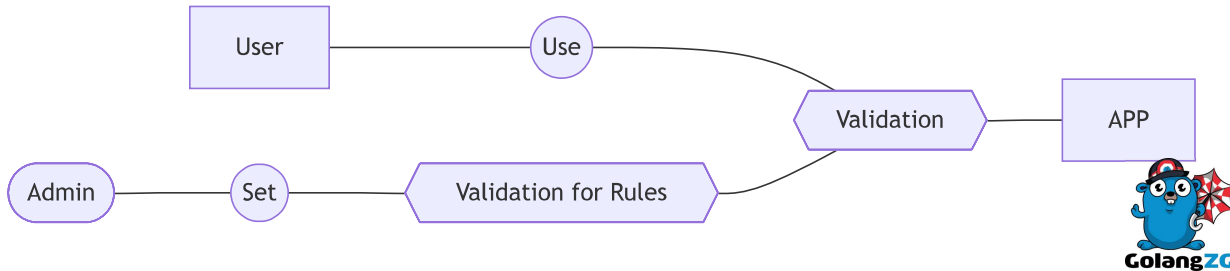
---

- [github.com/haproxytech/kubernetes-ingress](https://github.com/haproxytech/kubernetes-ingress)
- Kubernetes CRD validation with OpenAPI v3 + CEL
  - Further customisation of what user can do was needed
  - [documentation/annotations-custom.md](#)



# Validation types - CEL - real world

- Have predefined types of data (duration, int, uint, bool, string, float, json)
- Have predefined scopes (e.g., `frontend`, `backend`, `specific backend`, ...)
- Predefined template variables (e.g., `{{.BACKEND}}`, `{{.NAMESPACE}}`, `{{.INGRESS}}`, `{{.SERVICE}}`, `{{.POD_NAME}}`, `{{.POD_NAMESPACE}}`, `{{.POD_IP}}` )





# Validation types - CEL - validation for rules

```
timeout-server: # name of annotation
  section: all # can be all, fronted, backend (default)
  namespaces: # we can limit namespace usage
    - haproxy-controller
    - default
  resources: # limit usage to Service, Frontend or Backend na
    - <name>
  ingresses: # limit usage to specific ingresses
    - <name>
  order_priority: 100 # order of custom annotations in config
  template: "timeout server {{.}}" # template we can use
  type: duration # expected data type for conversion
  rule: "value > duration('42s') && value <= duration('42m', 's')"
```



# Validation types - CEL - usage

- rule

```
maxconn:
```

```
  type: int
```

```
  rule: "value >= 10 && value <= 1000000"
```

- usage

```
backend.example.com/maxconn: "1000"
```

# Validation types - CEL - usage

- rule

```
timeouts:
  section: backend
  template: |
    timeout server {{.server}}
    timeout server-fin {{.server_fin}}
    timeout tarpit {{.tarpit}}
  type: json
  rule: |
    'server' in value && value.server.matches('^[\0-9]+[smh]?$') &&
    'server_fin' in value && value.server_fin.matches('^[\0-9]+[smh]?$') &&
    'tarpit' in value && value.tarpit.matches('^[\0-9]+[smh]?$')
```

- usage

```
backend.example.com/timeouts: |
{
  "server": "42s",
  "server_fin": "10s",
  "tarpit": "5s"
}
```





# Validation types - CEL - source code

---

- [validations.go](https://github.com/google/validate)
- define rules and validation logic
- precompile CEL expressions for performance
- validation and result function (templating)
- gracefully handle errors and provide feedback to users
  - on validation failure, the user gets a clear error message indicating which rule was violated and why, helping them correct their input.
  - there is no direct response if something goes wrong
    - kubectl usage, but also, automation is often used for generation
    - error appears in configuration file, making it easier for administrators to identify and fix issues. (specific use-case)



